

Reviewer Pack — Minimal Rank of Free Lie Algebras and Resolution Proof Width

Entry 5418d08b5fb0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 14:33:43 UTC

Minimal Rank of Free Lie Algebras and Resolution Proof Width

Entry ID: 5418d08b5fb0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 14:33:43 UTC

1. Conjecture

Field A (mathematical branch): Lie Theory (Free Lie Algebras) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every conjunctive normal form (CNF) φ with n variables, the minimal rank of the free Lie algebra generated by the set of variables $\{x_1, \dots, x_n\}$ in the free Lie algebra on generators $\{x_1, \dots, x_n\}$, denoted as $r(\varphi)$, is at most a constant multiple of the resolution proof width $w(\varphi)$, i.e., $r(\varphi) \leq c \cdot w(\varphi)$ for some constant c .

Rationale (proposer’s reasoning):

Free Lie algebras provide an algebraic structure that can encode combinatorial information about complexity. The minimal rank of a free Lie algebra may capture structural properties of the CNF that contribute to its resolution proof width, revealing a potential bridge between algebraic structures and complexity theory.

Taxonomy category: Lie Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8fb4e6a578e30deb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated CNFs φ with n variables ($n \leq 40$), the ratio $r(\varphi)/w(\varphi)$ is below or equal to a constant $c \pm 3\%$ across at least 80% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - free lie algebra AND minimal rank AND resolution proof complexity - CNF AND free Lie algebra AND resolution width - Lie theory AND Boolean satisfiability AND proof width of CNFs

Top relevant hits considered: - [http://arxiv.org/abs/quant-ph/0301144v2] The Lie Algebra Rank Condition for Non-Bilinear Quantum System - [http://arxiv.org/abs/0810.4836v3] Simplicial complexes and minimal free resolution of monomial algebras - [http://arxiv.org/abs/2001.07570v2] Hom 3-Lie-Rinehart Algebras - [http://arxiv.org/abs/1310.8120v1] On the Tractability of Minimal Model Computation for Some CNF Theories - [http://arxiv.org/abs/1503.01613v3] Space proof complexity for random 3-CNFs - [http://arxiv.org/abs/1411.1619v1] Space proof complexity for random 3-CNFs via a $(2 - \varepsilon)$ -Hall's Theorem - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/1212.5380v2] On properties of principal elements of Frobenius Lie algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def dpll(cnf, assignment):
        if not cnf:
            return True
        unit_clause = next((c for c in cnf if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            new_assignment = assignment.copy()
            new_assignment[-abs(literal)] = literal > 0
            if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
                return True
            else:
                del new_assignment[-abs(literal)]
        pure_literal = next((l for l in range(1, n + 1) if all(l not in c or -l not in c for c in
```

```

    if pure_literal is not None:
        new_assignment[pure_literal] = True
        if dpll([c for c in cnf if pure_literal not in c and -pure_literal not in c], new_assignment):
            return True
        else:
            del new_assignment[pure_literal]
    literal = random.choice(range(1, n + 1))
    new_assignment[literal] = True
    if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
        return True
    else:
        new_assignment[literal] = False
        if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
            return True
        else:
            return False

def resolution(cnf):
    clauses = set(tuple(sorted(c)) for c in cnf)
    while True:
        new_clauses = []
        for c1, c2 in itertools.combinations(clauses, 2):
            if len(set(c1) & set(c2)) == 1:
                literal = next(l for l in set(c1) if l not in set(c2))
                new_clause = tuple(sorted([l for l in c1 + c2 if l != -literal]))
                if new_clause not in clauses:
                    new_clauses.append(new_clause)
        if not new_clauses:
            return len(clauses)
        clauses.update(new_clauses)

def free_lie_algebra_rank(cnf):
    n = max(abs(l) for clause in cnf for l in clause)
    generators = list(range(1, n + 1))
    relations = []
    for clause in cnf:
        for i in range(len(clause)):
            for j in range(i + 1, len(clause)):
                relations.append((clause[i], clause[j]))
    rank = 0
    while relations:
        relation = relations.pop(0)
        if relation[0] != -relation[1]:
            rank += 1
            new_relations = []
            for r in relations:
                if r[0] == relation[0] or r[1] == relation[0]:
                    new_relations.append((r[0], -relation[1]))
                elif r[0] == relation[1] or r[1] == relation[1]:
                    new_relations.append((-relation[0], r[1]))
            relations = new_relations
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

```

```

for n in n_values:
    for _ in range(5):
        cnf = generate_cnf(n)
        w_phi = resolution(cnf)
        r_phi = free_lie_algebra_rank(cnf)
        if w_phi == 0 or r_phi == 0:
            continue
        results.append((n, w_phi, r_phi))

if not results:
    return {
        "metric_name": "r_phi_over_w_phi",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n_max = max(n for n, _, _ in results)
instances_tested = len(results)
r_phi_over_w_phi_values = [r_phi / w_phi for _, w_phi, r_phi in results]
mean_r_phi_over_w_phi = sum(r_phi_over_w_phi_values) / instances_tested
std_r_phi_over_w_phi = math.sqrt(sum((x - mean_r_phi_over_w_phi) ** 2 for x in r_phi_over_w_phi_values))

return {
    "metric_name": "r_phi_over_w_phi",
    "metric_value": mean_r_phi_over_w_phi,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": all(0.97 <= x / y <= 1.03 for _, y, x in results),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

    results = [run_trial(seed)["metric_value"] for seed in seeds if run_trial(seed)["metric_value"] is not None]
    support_fraction = sum(1 for r in results if 0.97 <= r / max(results) <= 1.03) / len(results)

    if all(r is not None for r in results):
        RESULT = "SUPPORTED" if support_fraction >= 0.8 else "FALSIFIED"
        print(f"{RESULT} mean={sum(results)/len(results):.2f} std={math.sqrt(sum((x - sum(results)/len(results))**2)/len(results)):.2f}")
    elif any(r is None for r in results):
        RESULT = "INCONCLUSIVE" if all(r is not None for r in results) else "FALSIFIED"
        print(f"{RESULT} counterexample=\"mapping_undefined\" first_failing_seed=1")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17758
2	preregistration	ollama_remote	glm4:latest	0	0	9366
3	novelty	ollama_remote	glm4:latest	0	0	11684
4	novelty	ollama_remote	glm4:latest	0	0	9874
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23173
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9203
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21845
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16431
9	judge	ollama_remote	glm4:latest	0	0	17322

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 136655 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/5418d08b5fb0.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/5418d08b5fb0.jsonl](#) for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: [research/pvsnp_sandbox/test_*.py](#) (per-cycle)

Replay tarball: [research/replay/5418d08b5fb0.tar.gz](#) (if generated)